

Sbt

Size Balanced Tree

Size Balanced Tree (SBT) 是由中国 OI 选手陈启峰在 2007 年提出的一种自平衡二叉搜索树 (Self-Balanced Binary Search Tree, SBBST), 通过检查子树的节点数量进行自身的平衡维护。相比于红黑树, AVL 等主流自平衡二叉搜索树而言, Size Balanced Tree 支持在 $O(\log n)$ 的时间复杂度内查询某个键值在树中的排名 (rank).

节点定义

相比与普通二叉搜索树, SBT 的每个节点 仅需要多维护一个整数字段 `size`, 用于储存以 为根的子树中节点的个数。节点类型 `Node` 的具体定义如下:

Identifier	Type	Description
<code>left</code>	<code>Node*</code>	左子节点引用
<code>right</code>	<code>Node*</code>	右子节点引用
<code>size</code>	<code>int</code>	以该节点为根的子树中节点的个数

性质

Size Balanced Tree 中任意节点 满足如下几条性质:

```
1 size(N.left) >= size(N.right.left)
2 size(N.left) >= size(N.right.right)
3 size(N.right) >= size(N.left.left)
4 size(N.right) >= size(N.left.right)
```

使用自然语言可描述为: 任意节点的 `size` 不小于其兄弟节点 (Sibling) 的所有子节点 (Nephew) 的 `size`.

平衡维护

旋转

SBT 主要通过旋转操作改变自身高度从而进行平衡维护。其旋转操作与绝大部分自平衡二叉搜索树类似, 唯一区别在于在完成旋转之后需要对旋转过程中左右子节点发生改变的节点更新 `size`. 示例代码如下:

```

1 void updateSize() {
2     USize leftSize = this->left != nullptr ? this->left->size : 0;
3     USize rightSize = this->right != nullptr ? this->right->size : 0;
4     this->size = leftSize + rightSize + 1;
5 }
6
7 static void rotateLeft(NodePtr& node) {
8     assert(node != nullptr);
9     // clang-format off
10    //      |                |
11    //      N                S
12    //      / \      l-rotate(N) / \
13    //      L  S      =====>  N  R
14    //      / \                / \
15    //      M  R                L  M
16    // clang-format on
17    NodePtr successor = node->right;
18    node->right = successor->left;
19    successor->left = node;
20
21    node->updateSize();
22    successor->updateSize();
23
24    node = successor;
25 }
26
27 static void rotateRight(NodePtr& node) {
28     assert(node != nullptr);
29     // clang-format off
30    //      |                |
31    //      N                S
32    //      / \      r-rotate(N) / \
33    //      S  R      =====>  L  N
34    //      / \                / \
35    //      L  M                M  R
36    // clang-format on
37    NodePtr successor = node->left;
38    node->left = successor->right;
39    successor->right = node;
40
41    node->updateSize();
42    successor->updateSize();
43
44    node = successor;
45 }

```

维护

Case 1

```
size(N.left) < size(N.right.left)
```

```

1 if (size(node->right->left) > size(node->left)) {
2   // clang-format off
3   //      |                |                |
4   //      N                N                [M]
5   //      / \      r-rotate(R) / \      l-rotate(N) / \
6   //      &LTL> R      =====> &LTL> [M]      =====> N      R
7   //      /                \                /
8   //      [M]                R                &LTL>
9   // clang-format on
10  rotateRight(node->right);
11  rotateLeft(node);
12  fixBalance(node->left);
13  fixBalance(node->right);
14  fixBalance(node);
15  return;
16 }

```

Case 2

size(N.left) < size(N.right.right)

```

1 if (size(node->right->right) > size(node->left)) {
2   // clang-format off
3   //      |                |
4   //      N                R
5   //      / \      l-rotate(N) / \
6   //      &LTL> R      =====> N      [M]
7   //      \                /
8   //      [M]                &LTL>
9   // clang-format on
10  rotateLeft(node);
11  fixBalance(node->left);
12  fixBalance(node);
13  return;
14 }

```

Case 3

size(N.right) < size(N.left.left)

```

1 if (size(node->left->left) > size(node->right)) {
2   // clang-format off
3   //      |                |
4   //      N                L
5   //      / \      r-rotate(N) / \
6   //      L &LTR>      =====> [M] N
7   //      /                \
8   //      [M]                &LTR>
9   // clang-format on
10  rotateRight(node);
11  fixBalance(node->right);
12  fixBalance(node);
13  return;
14 }

```

Case 4

size(N.right) < size(N.left.right)

```

1 if (size(node->left->right) > size(node->right)) {
2   // clang-format off
3   //      |                |                |
4   //      N                N                [M]
5   //      / \      l-rotate(L) / \      r-rotate(N) / \
6   //      L  &LTR>  =====> [M] &LTR>  =====> L   N
7   //      \                /                \
8   //      [M]                L                &LTR>
9   // clang-format on
10  rotateLeft(node->left);
11  rotateRight(node);
12  fixBalance(node->left);
13  fixBalance(node->right);
14  fixBalance(node);
15  return;
16 }

```

操作

插入

SBT 的插入操作需要在完成普通二叉搜索树的插入操作的基础上递归地进行节点 `size` 字段的更新及平衡维护。示例代码如下：

```

1 if (compare(key, node->key)) {
2   /* key < node->key */
3   if (node->left == nullptr) {
4     node->left = Node::from(key, value);
5     node->updateSize();
6   } else {
7     insert(node->left, key, value, replace);
8     node->updateSize();
9     fixBalance(node);
10  }
11 } else {
12   /* key > node->key */
13   if (node->right == nullptr) {
14     node->right = Node::from(key, value);
15     node->updateSize();
16   } else {
17     insert(node->right, key, value, replace);
18     node->updateSize();
19     fixBalance(node);
20   }
21 }

```

删除

根据 Size Balanced Tree 的提出者陈启峰在其论文中对于删除操作的描述：

It can result in a destroyed SBT. But with the insertion above, a BST is still kept at the height of where is the total number of insertions, not the current size.

删除操作虽然有可能使得 SBT 的性质被打破，但并不会使树的高度增高，因此不会影响后续操作的效率。但在实际情况下，如果在一次批量插入操作后只进行大量的删除和查询操作，依然有可能由于树的失衡影响整体效率，因此本文在实现 SBT 的删除操作时依然选择加入平衡维护。参考代码如下：

```

1 bool remove(NodePtr& node, K key, NodeConsumer action) {
2   assert(node != nullptr);

```

```

3
4  if (key != node->key) {
5      if (compare(key, node->key)) {
6          /* key < node->key */
7          NodePtr& left = node->left;
8          if (left != nullptr && remove(left, key, action)) {
9              node->updateSize();
10             fixBalance(node);
11             return true;
12         } else {
13             return false;
14         }
15     } else {
16         /* key > node->key */
17         NodePtr& right = node->right;
18         if (right != nullptr && remove(right, key, action)) {
19             node->updateSize();
20             fixBalance(node);
21             return true;
22         } else {
23             return false;
24         }
25     }
26 }
27
28 assert(key == node->key);
29 action(node);
30
31 if (node->isLeaf()) {
32     // Case 1: no child
33     node = nullptr;
34 } else if (node->right == nullptr) {
35     // Case 2: left child only
36     // clang-format off
37     //      P
38     //      |  remove(N)  P
39     //      N  =====>  |
40     //      /                L
41     //      L
42     // clang-format on
43     node = node->left;
44 } else if (node->left == nullptr) {
45     // Case 3: right child only
46     // clang-format off
47     //      P
48     //      |  remove(N)  P
49     //      N  =====>  |
50     //      \                R
51     //      R
52     // clang-format on
53     node = node->right;
54 } else if (node->right->left == nullptr) {
55     // Case 4: both left and right child, right child has no left child
56     // clang-format off
57     //      |                |
58     //      N  remove(N)    R
59     //      / \  =====>  /
60     //      L  R                L
61     // clang-format on
62     NodePtr right = node->right;
63     swapNode(node, right);
64     right->right = node->right;
65     node = right;
66     node->updateSize();
67     fixBalance(node);

```

```

68 } else {
69 // Case 5: both left and right child, right child is not a leaf
70 // clang-format off
71 // Step 1. find the node N with the smallest key
72 // and its parent P on the right subtree
73 // Step 2. swap S and N
74 // Step 3. remove node N like Case 1 or Case 3
75 // Step 4. update size for all nodes on the path
76 // from S to P
77 //      |              |
78 //      N              S              |
79 //      / \            / \            S
80 // L .. swap(N, S) L .. remove(N) / \
81 //      | =====> | =====> L ..
82 //      P              P              |
83 //      / \            / \            P
84 //      S ..          N ..          / \
85 //      \              \              R ..
86 //      R              R
87 //
88 // clang-format on
89
90 std::stack<NodePtr> path;
91
92 // Step 1
93 NodePtr successor = node->right;
94 NodePtr parent = node;
95 path.push(node);
96
97 while (successor->left != nullptr) {
98     path.push(successor);
99     parent = successor;
100    successor = parent->left;
101 }
102
103 // Step 2
104 swapNode(node, successor);
105
106 // Step 3
107 parent->left = node->right;
108 // Restore node
109 node = successor;
110
111 // Step 4
112 while (!path.empty()) {
113     path.top()->updateSize();
114     path.pop();
115 }
116 }
117
118 return true;
119 }

```

值得注意的是，在上述代码的 Case 5 中使用后继节点（也可以选择前驱节点）替换待删除节点 并删除替换后的 以后，需要更新替换前 节点的父节点 到替换后的 节点这条路径（如代码中注释所示）上的所有节点的 `size` 字段。本文的实现选择使用栈依次记录路径上的节点，最后再按遍历的相反顺序出栈进行更新。

查询排名

由于 SBT 节点中储存了子树节点个数的信息，因此可以在 $O(\log n)$ 的时间复杂度下查询某个 `key` 的排名（或者大于/小于某个 `key` 的节点个数）。示例代码如下：

```
1 USize countLess(ConstNodePtr node, K key, bool countEqual = false) const {
2     if (node == nullptr) {
3         return 0;
4     } else if (key < node->key) {
5         return countLess(node->left, key, countEqual);
6     } else if (key > node->key) {
7         return size(node->left) + 1 + countLess(node->right, key, countEqual);
8     } else {
9         return size(node->left) + (countEqual ? 1 : 0);
10    }
11 }
12
13 USize countGreater(ConstNodePtr node, K key, bool countEqual = false) const {
14     if (node == nullptr) {
15         return 0;
16     } else if (key < node->key) {
17         return size(node->right) + 1 + countGreater(node->left, key, countEqual);
18     } else if (key > node->key) {
19         return countGreater(node->right, key, countEqual);
20     } else {
21         return size(node->right) + (countEqual ? 1 : 0);
22     }
23 }
```

参考代码

下面的代码是用 SBT 实现的 `Map`，即有序不可重映射：

► 完整代码

本页面最近更新：2023/5/21 15:58:56, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [aofall](#), [CoelacanthusHex](#), [lr1d](#), [Marcythm](#), [Persdre](#), [Rlvance](#), [shuzhouliu](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用